2048 as a Markov Decision Problem: exact dynamic programming on small boards and reinforcement learning on the real game

Project work — Dynamic Programming and Reinforcement Learning, A.Y. 2024/25

Edoardo Caproni (141794) Tommaso Quintabà ([matricola])

November 11, 2026

Abstract

2048 is a single-player sliding-tile puzzle whose rules define a Markov decision problem with a known model. We solve small versions of the game exactly by dynamic programming: the game is a total-cost problem with termination states, value iteration reaches the exact solution after finitely many iterations, and since the sum of the tiles is a clock, ordering the states by it turns value iteration into a single backward recursion. We use the exact solution to measure how far the usual rules of thumb are from optimal play and how much of the outcome is luck. We then drop the knowledge of the model and learn to play from a simulator with the methods of the course, evaluating every learned value function and policy exactly against the dynamic-programming solution, which turns learning curves into exact measurements. In prediction, Monte Carlo beats TD(0) on this long game with a sparse reward, and intermediate n-step TD beats both. In control on the 2×3 board the corner rule wins 0.07% of the games, the optimal policy 93.9%, tabular Q-learning 88% and Sarsa with the action values written through the afterstates 92%; on the original 4×4 board ($\approx 10^{16}$ states) episodic semi-gradient Sarsa with linear n-tuple features reaches the 2048 tile in 93% of the games and 4096 in 81% after a million training games, with a mean score thirty times that of the heuristics. A deep Q-network built as in the PyTorch tutorial, the final test, learns the small board faster than any table but, after 10 000 games of the real game, still scores less than the rules of thumb. We close with what is known about the game itself: whether it can always be won (open; the best lower bound is 0.99969), whether an optimal strategy is known (no; the problem is NP-hard on general boards), and which heuristics are good (search and learned n-tuple evaluations, the method of this report).

1 Motivation

2048 (G. Cirulli, 2014 [3]) is a single-player puzzle played on a 4×4 grid of numbered tiles. Every move is a deterministic slide of all the tiles followed by the random appearance of a new tile, so the game is a Markov decision problem whose model is completely known. Small versions of the game (fewer cells, a smaller target tile) can be solved exactly by dynamic programming. The original game cannot: its state space is far too large. The project is built on this gap.

- **DP part.** On small boards, compute the optimal policy exactly and use it to answer questions that are hard to settle by playing: how well can the game be played, how good are the rules of thumb used by human players, and how much of the outcome is luck.
- **RL part.** Remove the knowledge of the model and learn from a simulator, with the methods of the course: prediction and tabular control on a small board, where the exact solutions of the DP part measure the learning algorithms, and semi-gradient Sarsa with a linear approximation on the original 4×4 board, where only approximate methods are feasible. A deep Q-network, built as in the PyTorch tutorial, closes the part as a final test.

2 The game

The game is played on a board of $R \times C$ cells with a target tile 2^K. In the original game $R = C = 4$, $K = 11$ (tile 2048) and $p_4 = 0.1$.

1. A move $u \in \{\mathsf{down}, \mathsf{left}, \mathsf{right}, \mathsf{up}\}$ slides all the tiles as far as possible in that direction. Two tiles with the same value that collide merge into one tile carrying their sum. A tile created by a merge cannot merge again in the same move, and in each line the merges are resolved starting from the side the tiles move to: moving left, 2 2 2 2 becomes 4 4. The score increases by the value of every tile created by a merge.

2. A move is legal only if it changes the board.

3. After every legal move a new tile appears in an empty cell chosen uniformly at random: a 2 with probability $1 - p_4$, a 4 with probability p_4. The game starts with two such tiles in two distinct cells.

4. The game is won when a move creates the target tile, and lost when no legal move exists.

3 Formulation as a Markov decision problem

State. A board is encoded by the exponents of its tiles, $x = (x_1, \ldots, x_{RC})$ in row-major order, with $x_i = 0$ for an empty cell and $x_i = k$ for a tile 2^k. The state x_t is the board observed before the t-th move, that is, after the random tile. The initial state x_0 has distribution p_0: two tiles in two distinct cells chosen uniformly at random, each a 2 with probability $1 - p_4$ or a 4 with probability p_4. Since the game ends as soon as the target is created, $x_i \in \{0, \ldots, K-1\}$, and the state space $\mathcal{X}$ is the set of boards reachable from p_0. Two kinds of terminal situations exist: the win, reached when a move creates 2^K, and the boards with no legal move (game over).

Control. The admissible controls are the legal moves,

$$\mathcal{U}(x) = \{u \in \{\mathsf{down}, \mathsf{left}, \mathsf{right}, \mathsf{up}\} : M(x, u) \neq x\},$$

where M is the slide/merge map defined below; the game is over exactly when $\mathcal{U}(x) = \emptyset$.

Deterministic part of a move. $y = M(x, u)$ is the board after the slide/merge and before the new tile, called the *afterstate*. M acts line by line: each row (for $\mathsf{left}/\mathsf{right}$) or column (for $\mathsf{up}/\mathsf{down}$), read towards the side of the move, is compacted by removing the empty cells, adjacent equal pairs are merged from the front with each tile used at most once, and the line is padded with empty cells. The points of the move, $r(x, u)$, are the sum of the values of the merged tiles.

Disturbance. $w_t = (i_t, v_t)$, where i_t is uniform over the set $E(y_t)$ of empty cells of the afterstate $y_t = M(x_t, u_t)$, and $v_t \in \{1, 2\}$ is the exponent of the new tile, with $P(v_t = 1) = 1 - p_4$ and $P(v_t = 2) = p_4$. Given y_t, w_t is independent of the past; its distribution depends on (x_t, u_t) only through y_t.

Dynamics. Writing $y + v\, e_i$ for the board y with exponent v placed in the empty cell i,

$$x_{t+1} = f(x_t, u_t, w_t) = M(x_t, u_t) + v_t\, e_{i_t}.$$

Objective. Maximise the probability of creating the target tile,

$$V^*(x) = \max_{\pi} P^{\pi}(\text{the target tile is created} \mid x_0 = x).$$

In the cost form used in the course [2], this is a problem with no stage cost and a terminal cost equal to 1 for a lost game and 0 for a won game; its optimal cost is $J^*(x) = 1 - V^*(x)$, the minimum probability of losing. A second objective is studied for comparison: the expected score, with stage reward $r(x_t, u_t)$ and no stop at the target tile.

Bellman equation. With the afterstate value W^*,

$$V^*(x) = \begin{cases} 0 & \text{if } \mathcal{U}(x) = \emptyset, \\ \max\limits_{u \in \mathcal{U}(x)} W^*(M(x,u)) & \text{otherwise,} \end{cases} \tag{1}$$

$$W^*(y) = \begin{cases} 1 & \text{if } \max_i y_i = K, \\ \sum\limits_{i \in E(y)} \dfrac{1}{|E(y)|} \Big[(1-p_4)\, V^*(y + e_i) + p_4\, V^*(y + 2e_i) \Big] & \text{otherwise.} \end{cases} \tag{2}$$

For the expected score, the maximisation becomes $\max_u \big[r(x,u) + W(M(x,u)) \big]$ and the first case of (2) disappears.

4 Structural properties

Proposition 1 (the tile sum is a clock)**.** *Let $\sigma(x) = \sum_{i:\, x_i > 0} 2^{x_i}$ be the sum of the tiles. For every board x, legal move u and disturbance $w = (i, v)$,*

$$\sigma\big(M(x,u)\big) = \sigma(x), \qquad \sigma\big(f(x,u,w)\big) = \sigma(x) + 2^v \in \{\sigma(x) + 2,\ \sigma(x) + 4\}.$$

Proof. Sliding moves tiles without changing their values, and a merge replaces two tiles 2^k with one tile $2^{k+1} = 2^k + 2^k$. The new tile adds 2^v. □

Corollary 1. *(i) The tile sum strictly increases along every trajectory, so no state is visited twice: the transition graph of the reachable states is acyclic.*

(ii) Before the target is created every tile is at most 2^{K-1}, so $\sigma(x_t) \le RC \cdot 2^{K-1}$. Since σ grows by at least 2 per move, every game ends within $RC \cdot 2^{K-2}$ moves, whatever the policy. All the policies are proper, and (1)–(2) has a unique solution.

(iii) Let $\mathcal{X}_\sigma = \{x \in \mathcal{X} : \sigma(x) = \sigma\}$. Transitions only go from $\mathcal{X}_\sigma$ to $\mathcal{X}_{\sigma+2} \cup \mathcal{X}_{\sigma+4}$. Hence V^*, which value iteration computes in a finite number of iterations (Section 5.1), is also computed exactly by a single backward recursion over σ, like the finite-horizon DP algorithm with stage index $\sigma/2$.

Total-cost formulation. In the terms of the course [2], 2048 is an infinite-horizon problem with total cost and two kinds of termination states, the win and the boards without legal moves, made absorbing and cost-free (like the destination of the shortest-path example, with a fictitious self-loop). Corollary 1(ii) gives the two assumptions of the total-cost case: termination states exist (A2) and every policy reaches them from every state (A3), here with probability one and within $RC \cdot 2^{K-2}$ moves. The Bellman equation (1)–(2) therefore has a unique solution, and value iteration converges to it from $V^{(0)} = 0$ with no discount.

Remark (afterstates). A move factors into a deterministic, controlled part $x \mapsto y = M(x,u)$ and a random, uncontrolled part $y \mapsto x'$. The expectation over the disturbance can therefore be computed in two steps, as in the slides of the course: first the afterstate y of every move, then the average over the new tile in (2). The same decomposition gives the RL part an exact parametrisation of the action values, $q(x,u) = r(x,u) + V(M(x,u))$ (Section 6.3).

Remark (symmetries). Rotations and reflections of the board (8 for a square board, 4 for a rectangular one) map the problem onto itself, with the moves mapped accordingly, so V^* is invariant under them. The DP does not exploit them and uses them as a correctness test; the features of the RL part share their weights among the symmetric images, so that the learned values are invariant too.

5 Exact dynamic programming

Algorithm 1 Exact DP by stages

Forward pass: starting from the initial distribution (two tiles), generate the reachable sets $\mathcal{X}_\sigma$ for increasing σ
for $\sigma = \sigma_{\max},\ \sigma_{\max} - 2,\ \ldots,\ \sigma_{\min}$ **do** ▷ backward pass
  for each $x \in \mathcal{X}_\sigma$ **do**
    compute $W^*(M(x,u))$ for every $u \in \mathcal{U}(x)$ with (2) ▷ uses V^* on $\mathcal{X}_{\sigma+2} \cup \mathcal{X}_{\sigma+4}$
    $V^*(x) \leftarrow \max_u W^*(M(x,u))$, $\mu^*(x) \leftarrow$ a maximising u ▷ $V^*(x) = 0$ if $\mathcal{U}(x) = \emptyset$
  end for
end for
return μ^* and the value at the start $\sum_x p_0(x)\, V^*(x)$

Replacing the maximisation with the action chosen by a fixed policy gives the exact value of that policy (policy evaluation), which is used for the benchmark policies and, in the RL part, for the learned policies. Ties are broken in the fixed order down, left, right, up. The cost is proportional to $\sum_\sigma |\mathcal{X}_\sigma| \cdot |\mathcal{U}| \cdot 2RC$.

5.1 Value iteration and the backward recursion

Value iteration is the algorithm of the course for the total-cost problem: starting from $V^{(0)} = 0$, every iteration applies the Bellman operator to all the states,

$$V^{(i+1)}(x) = \max_{u \in \mathcal{U}(x)} W^{(i)}(M(x,u)),$$

where $W^{(i)}$ is (2) with $V^{(i)}$ in place of V^*, and stops when $\max_x |V^{(i+1)}(x) - V^{(i)}(x)| \le \delta$. Started from zero, $V^{(i)}(x)$ is the optimal probability of winning within i moves. Since every game ends within a bounded number of moves, the iteration is exact after finitely many steps, at most one per stage, and not only in the limit. Table 1 confirms it with $\delta = 0$: on every board value iteration stops at a fixed point that coincides with the backward recursion to the last bit, after 46 iterations on the 2×3 board with target 64 (55 stages) and after a number of iterations below the number of stages on the other boards. Figure 1 shows how: $V^{(i)}$ at the start of the game, the probability of winning within i moves, is zero until $i \approx 27$ and reaches V^* at $i \approx 35$, the length of the optimal games (32.7 moves on average, Section 5.4); the largest error over the states stays close to 1 for 30 iterations and then falls to zero in 16.

The backward recursion of Algorithm 1 is the same iteration with the states updated in place (Gauss–Seidel) in decreasing tile sum: every update then uses values of later stages that are already final, so the first sweep gives V^* and the second only confirms it (Table 1). It does the work of one iteration instead of 47 on the 2×3 board, about 55 times less time; this is why the larger boards of Table 2 are solved with it.

Board	Objective	$\lvert\mathcal{X}\rvert$	Stages	Value iteration		$\max_x \lvert V^{\mathrm{VI}} - V^* \rvert$	Gauss–Seidel
				iterations	time (s)		sweeps
2×2, target 16	win	214	11	10	0.01	0	2
2×2, target 32	win	422	19	17	0.03	0	2
2×3, target 32	win	11 222	33	24	1.5	0	2
2×3, target 64	win	27 022	55	47	8.8	0	2
2×3, target 128	win	52 486	93	72	25.7	0	2
2×2	score	662	29	25	0.08	0	2
2×3	score	85 844	125	119	71.8	0	2

Table 1: Value iteration (Jacobi, from $V^{(0)} = 0$, stopping at $\max_x |V^{(i+1)} - V^{(i)}| = 0$) against the backward recursion: number of iterations including the last, confirming one; largest difference from the backward recursion over all the states; sweeps of the Gauss–Seidel version in decreasing tile sum. The backward recursion takes 0.16 s on the 2×3 board with target 64 and 0.51 s with the score objective. For the score objective the game continues past the target, so the target plays no role.

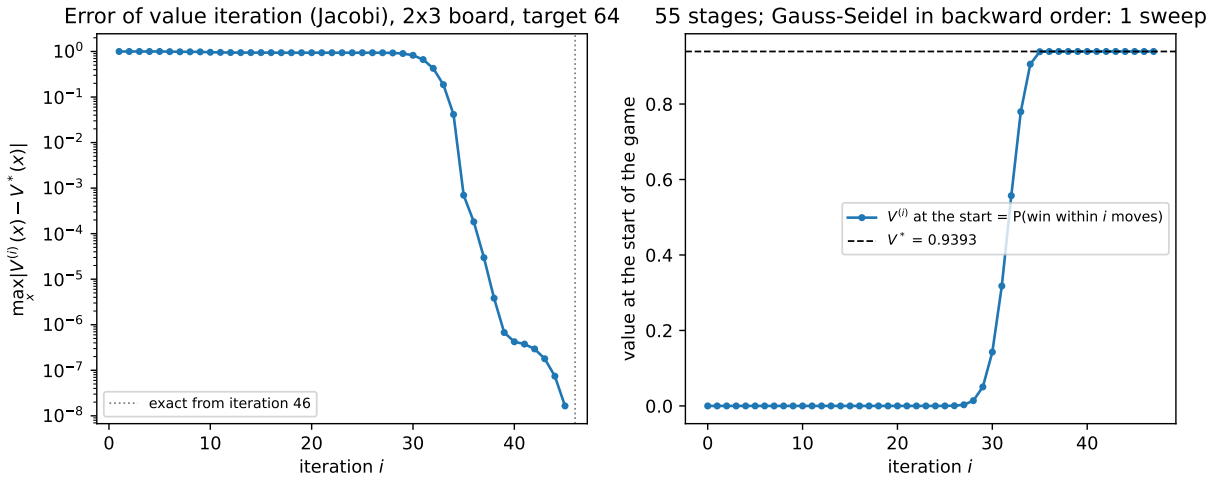


Figure 1: Value iteration on the 2×3 board, target 64. Left: largest error over the states (it is exactly zero from iteration 46). Right: value at the start of the game, i.e. the optimal probability of winning within i moves.

5.2 Benchmark policies

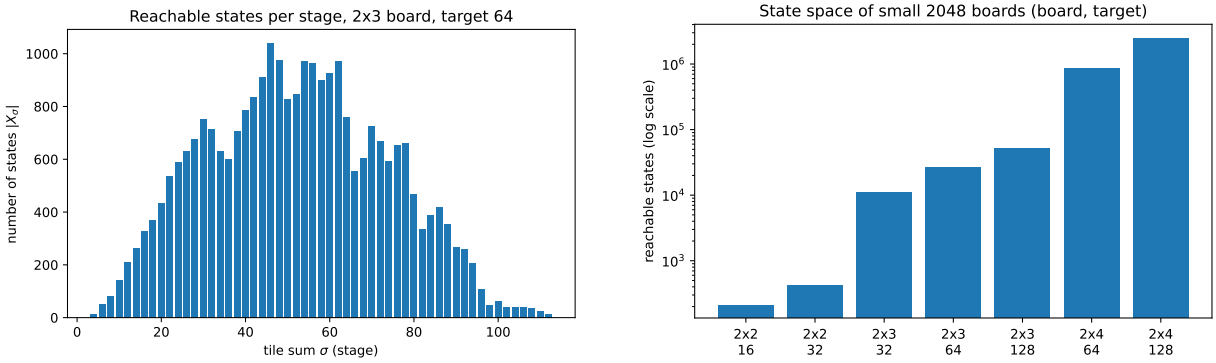
- **Corner rule**: the first legal move in the order down, left, right, up. It is the classic rule of thumb of keeping the large tiles in one corner and pressing up only when forced.
- **Greedy**: the legal move with the largest immediate score $r(x, u)$, ties broken in the same order.
- **Random**: a uniformly random legal move.

5.3 Size of the state space

Table 2 reports the number of reachable states, together with the naive bound K^{RC} (each cell holds an exponent in $\{0, \ldots, K-1\}$). Doubling the target roughly doubles the states. Adding cells is far more expensive: with target 64, the 2×4 board has about 30 times the states of the 2×3 board, and the 3×3 board about 180 times. For the original game the bound is $11^{16} \approx 4.6 \cdot 10^{16}$: exact DP is out of reach, which motivates the RL part. Figure 2a shows how the states of the 2×3 board are distributed over the stages.

Board	Target	$\lvert\mathcal{X}\rvert$	Bound K^{RC}	DP time	P(win)		
				(s)	optimal	corner	greedy
2×2	16	214	256	0.0	0.9625	0.2355	0.2474
2×2	32	422	625	0.0	0.0829	0.0000	0.0000
2×3	32	11 222	15 625	0.2	0.9999	0.2854	0.3365
2×3	64	27 022	46 656	0.3	0.9393	0.0007	0.0013
2×3	128	52 486	117 649	0.6	0.0570	0.0000	0.0000
2×4	64	881 260	1 679 616	100	0.99999	0.3611	0.3295
2×4	128	2 525 754	5 764 801	272	0.9973	0.0044	0.0037
3×3	64	4 820 069	10 077 696	240	1.0000	0.4341	0.4402
3×3	128	17 244 751	40 353 607	–	–	–	–
4×4	2048	intractable	$\approx 4.6 \cdot 10^{16}$	–	–	–	–

Table 2: Reachable decision states (target not yet created), naive bound, time of the backward recursion in pure Python, and exact win probability of the optimal policy and of two heuristics ($p_4 = 0.1$). The random policy wins with probability 0.1218, 0.0005, 0.0921, 0.0002, 0.0000, 0.0274, 0.0000 and 0.0921 on the first eight boards.



(a) States per stage, 2×3 board, target 64.

(b) Reachable states of the small boards.

Figure 2: The curse of dimensionality: the number of states grows exponentially with the number of cells.

5.4 Results on the 2×3 board with target 64

This board is the reference problem of the project: 27 022 states in 55 stages, solved in a fraction of a second, with an optimal win probability of 0.9393 that is neither trivially 0 nor 1. All the results below are exact unless stated otherwise.

The heuristics are hopeless. The corner rule wins 0.07% of the games, the greedy rule 0.13% and random play 0.02%, against 93.9% for the optimal policy. On this board the "keep the big tiles in a corner" advice is worthless: the optimal policy plays the corner move in only 49.5% of its decisions (weighted by how often each decision occurs under optimal play). Table 3 lists the departures from the corner rule that matter most, ranked by expected visits times loss of win probability. The boards tell one story: with six cells, what decides the game is not where the largest tile sits but whether a merge is still available after the next spawn. In the first, fourth and sixth boards the corner move keeps the 32 in place and fills the last free cell with tiles that do not match their neighbours, so that the game continues only if the spawn is a 4 (probability $p_4 = 0.1$, hence the loss of 0.9); the optimal move merges the same pair in the other direction and leaves a matching pair for the next move. In the second and fifth boards the optimal move takes

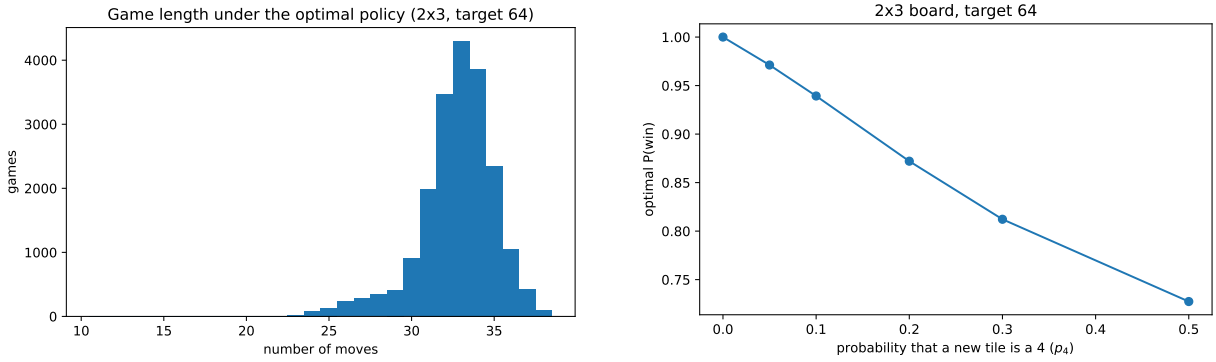
the largest tile *out* of its corner to line up a cascade ($4+4$, $8+8$, $16+16$, $32+32=64$ in the second board; the two 32s in the same column, an immediate win, in the fifth). In the third board the corner rule postpones a merge of two 8s that is available now. A rigid move order cannot see any of this; the optimal policy is not "anti-corner" either, it simply ranks a merge kept in reserve above a tile kept in place.

Importance	Board	Corner move	Optimal move	Loss
0.236	$\begin{smallmatrix}16&4&2\\32&8&2\end{smallmatrix}$	down	up	0.90
0.098	$\begin{smallmatrix}32&2&2\\16&8&4\end{smallmatrix}$	left	right	0.90
0.096	$\begin{smallmatrix}8&8&4\\32&\cdot&2\end{smallmatrix}$	down	left	0.86
0.086	$\begin{smallmatrix}16&2&\cdot\\32&8&2\end{smallmatrix}$	right	up	0.90
0.081	$\begin{smallmatrix}2&32&2\\32&4&4\end{smallmatrix}$	left	right	0.91
0.079	$\begin{smallmatrix}16&4&2\\32&2&2\end{smallmatrix}$	down	up	0.86

Table 3: Most important departures of the optimal policy from the corner rule, 2×3 board, target 64. Importance = expected number of visits of the state under the optimal policy times the loss; loss $= V^*(x) - Q^*(x, u_{\text{corner}})$, the win probability given up by the corner move.

Monte Carlo validation. Simulating 20 000 games with the optimal policy gives a win frequency of 0.9395 ± 0.0033 (95% confidence), in agreement with the exact value 0.9393. The games last 32.7 moves on average (Figure 3a).

The role of chance. Figure 3b shows the optimal win probability as a function of p_4. With $p_4 = 0$ (only 2s appear) the game is won with certainty; the 4s are what makes it lose, and at $p_4 = 0.5$ even the optimal player wins only 72.7% of the games. On this small board, therefore, luck decides at least 6% of the games whatever the skill.



(a) Game length under the optimal policy.

(b) Optimal $P(\text{win})$ as a function of p_4.

Figure 3: Monte Carlo check and sensitivity to the spawn distribution, 2×3 board, target 64.

Win probability or expected score? The same recursion with the merge points as stage reward maximises the expected score, which is what the game displays and what most 2048 programs optimise. The maximum expected score is 480.3. The score-optimal policy wins 93.87% of the games (against 93.93%) and agrees with the win-optimal move in 97.9% of the decisions. The two objectives are almost, but not exactly, aligned: the score rewards merging, and merging is what makes room for the target. This observation is used in the RL part, where the score is a much denser reward than the win.

6 Reinforcement learning

6.1 Setting

The agent now interacts only with a simulator of the game, with the interface `reset`/`step` of Gymnasium: it observes the board, chooses a legal move, and receives the next board and a reward. It does not know the spawn distribution. Two reward signals are used:

- **win**: +1 when a move creates the target tile, which ends the episode, and 0 otherwise. The return of an episode is the indicator of a win, so the value of a state is its win probability: $v_\pi = P^\pi(\text{win})$ and $v_* = V^*$ of Section 5.4.
- **score**: the points of each move; play continues past the target and the episode ends at game over. The value is the expected score.

By Corollary 1, every episode ends within a bounded number of moves and no state is visited twice in an episode, so the undiscounted return ($\gamma = 1$) is well defined and first-visit and every-visit Monte Carlo methods coincide.

The win reward is sparse: on the 2×3 board with target 64, random play wins 2 games in 10 000, so an agent that explores at random almost never observes a reward. The score reward is dense, and Section 5.4 showed that the score-optimal policy is almost win-optimal. The experiments therefore use both: the score reward to learn, and the exact win probability of the learned greedy policy, computed by the DP, to measure.

Exact learning curves. Because the DP of the small boards is available, the learned policies are not evaluated by simulation but exactly: the greedy policy of the agent is plugged into the backward recursion (policy evaluation) to obtain its win probability and expected score; the learned values are compared with V^* by the root mean square error weighted by the visits of the optimal policy; and the share of decisions (again weighted by the visits of the optimal policy) in which the greedy action is optimal is recorded. The curves have no Monte Carlo noise: all their variability comes from learning.

6.2 Prediction: TD(0), n-step TD and Monte Carlo

Before control, the prediction problem: estimate the value v_π of a fixed policy from its episodes alone. The policy is the optimal one of Part 1 and the reward is the win, so $v_\pi = V^*$ is the win probability, known exactly; the study is the one of the random-walk examples of the course [11, Example 6.2, §7.1], on a problem with 12 939 states visited by the policy instead of 5. Three methods with constant step size α and $\gamma = 1$ are compared:

- **TD(0)** (§6.1): $V(S_t) \leftarrow V(S_t) + \alpha\,[R_{t+1} + V(S_{t+1}) - V(S_t)]$;
- **n-step TD** (§7.1), with target $G_{t:t+n} = R_{t+1} + \cdots + R_{t+n} + V(S_{t+n})$, for $n = 2, 4, 8, 16$;
- **constant-α Monte Carlo** (§6.1), whose target is the full return: the limit $n \to \infty$ of n-step TD.

All the estimates start from $V_0 = 0.5$, as in Example 6.2. The error is $\sqrt{\overline{VE}}$ of §9.2: the root mean square error against V^* weighted by the on-policy distribution of the policy (its expected visits per game, normalised), the same quantity used for the control methods below. Each run simulates 20 000 games of the optimal policy, all the methods learn from the same games, and the results are averaged over 50 runs. Since no state repeats within a game, the updates of a game can be applied when it ends, in the order of the online algorithm, with the same result.

Results. Figure 4 reverses the ranking of the random walk, where TD(0) beats Monte Carlo. Here Monte Carlo is the fastest at the start and TD(0) the slowest at every budget: averaged over the first 1 000 games the best TD(0) error is 0.33 against 0.17 for Monte Carlo, over 20 000 games 0.114 against 0.085. The reason is the length of the games and the sparsity of the reward: the only nonzero reward is the win at the end of a 33-move game, and TD(0) moves its information back by one state per visit, through 12 939 states that are each visited rarely, whereas the Monte Carlo target carries the outcome to every state of the game at once. With a longer budget the bootstrapping pays off and the intermediate n win, as in Figure 7.2 of the book: over 20 000 games the best is 8-step TD ($\alpha = 0.2$, error 0.080), then 4-step and 16-step (0.082–0.083), Monte Carlo (0.085, $\alpha = 0.1$), 2-step (0.095) and TD(0) (0.114, $\alpha = 0.6$); with these step sizes the 8-step estimates are also the most accurate at the end (error 0.054 after 20 000 games, against 0.062 for Monte Carlo and 0.064 for TD(0)). The best step size decreases with n, because a longer return is a noisier target, and all methods degrade quickly above $\alpha \approx 0.6$. The same trade-off returns in control: the methods that propagate credit further per update learn faster (Section 6.3).

6.3 Tabular control on the 2×3 board

Six control methods of the course [11] are implemented, with ϵ-greedy exploration restricted to the legal moves:

- **Monte Carlo control** (on-policy, first visit, ϵ-soft; §5.4), with sample-average updates;
- **Sarsa** (§6.4), **Q-learning** (§6.5) and **Expected Sarsa** (§6.6), with constant step size α;
- **n-step Sarsa** (§7.2) with $n = 3$;
- **Sarsa on afterstates**: Sarsa (§6.4) with the action values written as $Q(x, u) = r(x, u) + V(M(x, u))$, so that all the pairs (x, u) that lead to the same afterstate share one entry $V(y)$. The parametrisation is exact, because $q_\pi(x, u) = r(x, u) + \mathbb{E}[v_\pi(x') \mid y = M(x, u)]$ depends on (x, u) only through $r(x, u)$ and y. The agent needs the slide rule M, but not the spawn distribution; the tic-tac-toe player of [11, §1.5] learns the values of the positions reached by its own moves in the same way. Since $R_{t+1} = r(S_t, A_t)$ cancels, the Sarsa error is

  $$R_{t+1} + Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) = r(S_{t+1}, A_{t+1}) + V(Y_{t+1}) - V(Y_t), \qquad Y_t = M(S_t, A_t),$$

  so the update of $Q(S_t, A_t)$ is $V(Y_t) \leftarrow V(Y_t) + \alpha\,[r(S_{t+1}, A_{t+1}) + V(Y_{t+1}) - V(Y_t)]$, with target 0 when S_{t+1} is game over; A_{t+1} is the next move actually chosen, so the method is on-policy. With a linear approximation of V this is the algorithm used on the 4×4 board (Section 6.4).

Every method is run for 200 000 episodes with $\alpha = 0.1$, $\epsilon = 0.1$ and 5 seeds, on three settings: score reward; win reward from zero initial values; win reward with optimistic initial values $Q = 1$ ([11, §2.6]). The greedy policy is evaluated exactly every 2 000 episodes. The greedy choice breaks ties in the order down, left, right, up, so an agent with all-zero values plays the corner rule.

Results. Figure 5 and Table 4 summarise the 90 runs. Every method learns to play far better than any heuristic from the simulator alone: after 200 000 episodes the exact win probability of the greedy policies lies between 0.68 and 0.87, against 0.0007 for the corner rule, and the expected score between 275 and 412 against a maximum of 480. None reaches the optimum 0.9393 within the budget: with constant $\alpha = 0.1$ and $\epsilon = 0.1$ the estimates keep fluctuating and the behaviour policy keeps making exploratory mistakes, which the on-policy methods learn to fear. The ranking is consistent with the theory of the course.

- **Q-learning is the best method on plain tables $Q(x, u)$ in every setting** (0.757, 0.816, 0.875), because it is off-policy: its target $\max_a Q(x', a)$ estimates q_* whatever the exploration, whereas Sarsa and Expected Sarsa estimate the value of the ϵ-greedy policy, which is lower.

The error of the values against V^* makes this visible: with the win reward and optimistic initialisation, the weighted RMSE of Q-learning is 0.021, that of Sarsa 0.158 and of Expected Sarsa 0.148 (Figure 5d), although their greedy policies are not far apart. Expected Sarsa beats Sarsa in every setting (lower variance of the target), and 3-step Sarsa beats both in two of three.

- **Sarsa on afterstates is the best or second best everywhere** (0.796, 0.799, 0.848) and learns fastest (Figure 5a–c) with the fewest parameters: one value per afterstate instead of four per state, 10 478 entries against 12 464 states seen by Q-learning with the score reward. The shared entries are what make it fast: every update of $V(y)$ improves the action value of all the pairs (x, u) that lead to y. It beats plain Sarsa, the same on-policy algorithm on a plain table, by 0.05–0.12 in every setting: the gain comes from the parametrisation alone.

- **Monte Carlo control learns fast at first and then slowly**: it leads during the first 20 000 episodes with the score reward, but ends last or second to last, because it does not bootstrap and the returns of 30-move episodes are noisy.

- **The sparse win reward is slower but not hopeless.** From zero initial values the agents play the corner rule at first (the tie-break of the greedy choice), win nothing for about 10 000 episodes, and then take off once the ϵ-greedy exploration has stumbled on a few wins whose value propagates backwards (Figure 5b). Optimistic initial values ($Q_0 = 1$) remove the dead start and leave no state unvisited (the share of the optimal policy's visits to unseen boards drops from 15–20% to 0), and they give the best final results for Q-learning and Sarsa on afterstates. The two other on-policy Sarsa methods gain less: under optimism the exploring behaviour is more erratic, and that is what they evaluate.

- **The score reward is a good surrogate**: the greedy policies learned from the score win 71–80% of the games although they never observed a win signal, in line with the 97.9% agreement of the two optimal policies measured in Section 5.4.

Step size and exploration. Figure 6 shows the exact win probability after 200 000 episodes of Q-learning and Expected Sarsa with the score reward over a grid of constant α and ϵ. Within this budget the larger step sizes win ($\alpha = 0.2$–0.4 give 0.80–0.83 against 0.75 for $\alpha = 0.1$ and 0.3–0.5 for $\alpha = 0.02$): the estimates are noisier but they reach useful values on the 12 000 visited states much sooner, and the greedy policy is evaluated, not the noisy Q. Exploration has an optimum in the middle ($\epsilon = 0.05$–0.2): too little and the agent locks on the first good moves, too much and the behaviour policy loses games that the on-policy methods then learn to avoid in the wrong way. Replacing the constants by decaying schedules (ϵ from 0.3 to 0.01 and α from 0.2 to 0.02, linearly over the run) does not help the one-step action-value methods within this budget (Figure 7): Q-learning ends at 0.72 instead of 0.76, Sarsa and Expected Sarsa at 0.70, because the heavy early exploration wastes episodes on positions the greedy policy never visits, and the late small step sizes freeze the estimates before they are accurate. The methods that propagate credit further per episode do profit: 3-step Sarsa rises from 0.71 to 0.76 and Sarsa on afterstates from 0.80 to 0.85, with an exact expected score of 430 out of 480. The lesson is the one of the course: the schedules that guarantee convergence in the limit (decreasing α, GLIE exploration) are not the fastest for a fixed budget, and the right choice depends on how far each update reaches.

6.4 Function approximation: semi-gradient Sarsa with n-tuple features

On the 4×4 board tabular methods are not feasible. The control method of the course with function approximation is episodic semi-gradient Sarsa [11, §10.1], the Sarsa update applied to a parametrised action value $\hat{q}(x, u; \mathbf{w})$:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R_{t+1} + \hat{q}(S_{t+1}, A_{t+1}; \mathbf{w}) - \hat{q}(S_t, A_t; \mathbf{w})\big]\nabla\hat{q}(S_t, A_t; \mathbf{w}).$$

	score reward			win reward, $Q_0 = 0$			win reward, $Q_0 = 1$		
Method	P(win)	score	agree	P(win)	agree	RMSE	P(win)	agree	RMSE
MC control	0.738	387	0.63	0.676	0.56	0.62	0.711	0.56	0.54
Sarsa	0.713	367	0.53	0.753	0.54	0.68	0.726	0.68	0.16
Q-learning	0.757	383	0.54	0.816	0.56	0.53	0.875	0.78	0.02
Expected Sarsa	0.737	376	0.53	0.787	0.55	0.66	0.761	0.69	0.15
3-step Sarsa	0.714	381	0.59	0.754	0.57	0.62	0.795	0.70	0.33
Sarsa on afterstates	0.796	412	0.63	0.799	0.60	0.62	0.848	0.71	0.30
optimal (DP)	0.939	480	1	0.939	1	0	0.939	1	0

Table 4: Tabular methods on the 2×3 board, target 64, after 200 000 episodes ($\alpha = 0.1$, $\epsilon = 0.1$, mean over 5 seeds; the standard deviation over seeds is at most 0.025). Exact win probability and expected score of the greedy policy, share of the optimal policy's decisions in which the greedy action is optimal, and weighted RMSE of the learned state values against V^* (win objective).

The action values are parametrised as in the tabular Sarsa on afterstates, with a linear approximation of V:

$$\hat{q}(x, u; \mathbf{w}) = r(x, u) + \hat{V}(M(x, u); \mathbf{w}), \qquad \hat{V}(y; \mathbf{w}) = \mathbf{w}^\top \boldsymbol{\phi}(y) = \sum_{m=1}^{M} w_m[\, \phi_m(y)\,],$$

where every feature m is a tuple of n cells of the board, $\phi_m(y) \in \{0, \dots, 15\}^n$ is the pattern of exponents found in those cells, and w_m is a lookup table with 16^n entries: the binary feature vector $\boldsymbol{\phi}(y)$ has exactly one active entry in every table, like M overlapping state aggregations ([11, §9.3]). These features are the *n-tuple networks* of Szubert and Jaśkowski [12]. The gradient of $\hat{q}(S_t, A_t; \mathbf{w})$ is $\boldsymbol{\phi}(Y_t)$, with $Y_t = M(S_t, A_t)$, and $R_{t+1} = r(S_t, A_t)$ cancels as in the tabular case, so the update adds the error, divided by M, to the M active weights:

$$w_m[\phi_m(Y_t)] \leftarrow w_m[\phi_m(Y_t)] + \frac{\alpha}{M}[r(S_{t+1}, A_{t+1}) + \hat{V}(Y_{t+1}) - \hat{V}(Y_t)], \qquad m = 1, \dots, M,$$

with target 0 when S_{t+1} is game over. The step size of the weights is α/M, so that α is the step of $\hat{V}$ whatever the number of features. The moves are chosen greedily with respect to $\hat{q}$ ($\epsilon = 0$, so that Sarsa, Q-learning and Expected Sarsa coincide): the random tile is the only source of exploration, as in the Mountain Car example of [11, §10.1], where the exploration came from the initial values. With *symmetric weight sharing*, every tuple is read in all the images of the board under its 8 symmetries (4 on a rectangular board), all sharing the same table, so that $\hat{V}$ is invariant under the symmetries, like V^*: this is a choice of features, not a change of algorithm.

Two tuple sets are used: **rows, columns and** 2×2 **squares** ($n = 4$; on the 4×4 board 5 tables of 16^4 entries after merging the symmetric ones, 327 680 weights, 40 active features), the architecture of [12]; and 2×3 **rectangles** ($n = 6$; 2 tables of 16^6 entries, 33.5 million weights, 16 active features), closer to the networks of [14, 5]. On the small boards the tuple set **full board** (one table indexed by the whole afterstate) is the tabular Sarsa on afterstates of Section 6.3 written as a network, and serves as a sanity check.

6.4.1 Validation on the small boards

Before the real game, the algorithm is run on the three small boards where the exact solution is known, so that the learned values and the learned policy can be compared with V^* and π^* exactly, as in Section 6.3. Every configuration is trained for 300 000 games with $\alpha = 0.1$ (3 seeds on 2×3, 2 on 2×4, 1 on 3×3) and the checkpoints are evaluated with the exact DP: win probability of the greedy policy, share of the optimal policy's decisions in which the greedy action is optimal, and weighted RMSE of $\max_u[r + \hat{V}(M(x, u))]$ against V^* under the optimal occupancy (win objective only; the score-objective DP is available on 2×3 only). Figure 8 and Table 5 summarise the runs.

Board	Tuples	Sym.	Reward	$\hat{V}_0$	P(win)	agree	RMSE
$2 \times 3/64$	full board (tabular)	–	win	1	**0.923**	0.85	0.016
	full board (tabular)	–	win	0	0.042	0.51	0.90
	rows, cols, squares	yes	win	1	0.845	0.80	0.10
	rows, cols, squares	no	win	1	0.839	0.71	0.22
	rows, cols, squares	yes	score	0	0.826	0.77	–
	full board (tabular)	–	score	0	0.007	0.52	–
	full board, $\epsilon = 0.05$	–	score	0	0.812	0.62	–
	full board (tabular)	–	score	480	0.856	0.73	–
	optimal / corner rule				0.939 / 0.001		
$2 \times 4/128$	rows, cols, squares	yes	win	0	0.860	0.62	0.12
	rows, cols, squares	yes	win	1	0.852	0.63	0.11
	rows, cols, squares	no	win	0	**0.870**	0.45	0.44
	2×3 rectangles	yes	win	0	0.833	0.58	0.19
	rows, cols, squares	yes	score	0	0.828	0.61	–
	2×3 rectangles	yes	score	0	0.867	0.60	–
	optimal / corner rule				0.997 / 0.004		
$3 \times 3/64$	rows, cols, squares	yes	win	0	0.986	0.89	0.08
	rows, cols, squares	no	win	0	0.995	0.92	0.10
	2×3 rectangles	yes	win	0	0.982	0.90	0.35
	rows, cols, squares	yes	score	0	0.982	0.93	–
	2×3 rectangles	yes	score	0	**0.998**	0.93	–
	optimal / corner rule				1.000 / 0.434		

Table 5: n-tuple networks on the small boards after 300 000 training games ($\alpha = 0.1$, greedy play unless stated): exact win probability of the greedy policy, share of optimal decisions, weighted RMSE of the values against V^* (win reward only). $\hat{V}_0$ is the initial value of every afterstate.

- **The tabular network converges, but only with exploration.** On 2×3 the full-board table with the win reward and optimistic initial values ($\hat{V}_0 = 1$) reaches a win probability of 0.923 against the optimum 0.939, with the values within 0.016 RMSE of V^* and still improving at 300 000 games: this is the tabular Sarsa on afterstates of Table 4 with a different exploration mechanism, and it gets closer to the optimum because optimism explores every move once and then follows the best, instead of wasting a fixed share ϵ of the moves forever. With $\hat{V}_0 = 0$ the same learner wins 0.7% of the games with the score reward and 4% with the win reward: a greedy learner that starts from zero locks on the first moves that ever gave a reward (the corner rule wins one game in 1 500 on this board), and the random tiles alone do not provide enough exploration to leave them. Either optimism ($\hat{V}_0 = 480$, the optimal score: 0.856) or ϵ-greedy play (0.812) repairs it; the latter learns the values of the ϵ-greedy policy, not of the greedy one, which shows in its RMSE (226 points against 33 for the optimistic learner).

- **Approximation error.** With the rows, columns and squares of the 2×3 board (two 3-cell rows, three 2-cell columns and two 2×2 squares: 4 tables after symmetry merging, 14 active features), the learner rises faster than the table (0.80 against 0.22 after 3 000 games, because every update is shared by all the afterstates with the same rows and columns) but plateaus at 0.83–0.85, a loss of 0.09 in win probability that no amount of training recovers: the exact measure of what these features cannot represent on this board. The score reward gives the same plateau (0.826), consistent with the 98% agreement of the two objectives found in Part 1.

- **Harder boards, no optimism needed.** On $2 \times 4/128$ the corner rule wins 0.4% of the games and on $3 \times 3/64$ 43%; the zero-initialised greedy learner climbs from there to 0.86 and to 0.99 respectively. Features that generalise are what the tabular learner lacked: the rare early wins update every afterstate that shares a row, a column or a square with the winning line, and

after 10 000 games the 2×4 agent already wins 60–70% of the games (optimism, $\hat{V}_0 = 1$, only speeds this up: 0.84 at 10 000 games). The 6-cell rectangles, with 16^6 entries per table, start much slower (0.13 after 10 000 games on 2×4) and are still rising at 300 000 games, where they are the best configuration with the score reward on both boards (0.867 and 0.998): more capacity, more games needed, the trade-off that the 4×4 experiment confirms on a larger scale. The remaining gap to the optimum on $2 \times 4/128$ (0.87 against 0.997) is of the same order as the plateau on 2×3.

- **Symmetric weight sharing** improves the values much more than the policy. On $2 \times 4/128$ it cuts the RMSE from 0.44 to 0.12 and raises the share of optimal decisions from 0.45 to 0.62, but the win probability of the two greedy policies is the same (0.870 and 0.860): a network without weight sharing only learns the orientations that its own play produces, and its values on the mirror images, which the optimal occupancy visits and the RMSE counts, are wrong, but its own policy never needs them. Weight sharing pays off as a four-fold reduction of the number of weights to learn, with the same accuracy, which is what matters on the 4×4 board.

6.4.2 The original 4×4 game

The two networks are trained with the score reward on the original game (4×4, $p_4 = 0.1$) for 300 000 games, with $\alpha \in \{0.05, 0.1, 0.25, 0.5\}$ for the 4-tuples and $\alpha \in \{0.1, 0.25, 0.5\}$ for the 6-tuples, two seeds each (14 runs, about one hour each on one core with the bitboard engine of the appendix). Every checkpoint is evaluated with learning switched off on 1 000 games with a fixed seed; the three heuristics are evaluated on 10 000 games with the same engine. Here "2048 reached" means that the tile appeared during the game, which continues until no move is left.

Features	α	Score	Moves	P(2048)	P(4096)	P(8192)	Weights
rect (6-tuples)	0.1	65 958	3 016	0.895	0.670	0.147	33.5 M
rect (6-tuples)	0.25	60 111	2 758	0.863	0.620	0.125	33.5 M
rect (6-tuples)	0.5	50 051	2 359	0.878	0.575	0.011	33.5 M
rcs (4-tuples)	0.05	44 060	2 115	0.788	0.462	0.004	0.33 M
rcs (4-tuples)	0.1	42 220	2 034	0.822	0.436	0.002	0.33 M
rcs (4-tuples)	0.25	30 662	1 550	0.764	0.144	0.000	0.33 M
rcs (4-tuples)	0.5	18 938	1 026	0.456	0.000	0.000	0.33 M
rect (6-tuples), 10^6 games	0.1	92 352	4 004	0.927	0.811	0.422	33.5 M
greedy heuristic	–	3 027	256	0.000	0.000	0.000	–
corner rule	–	2 564	227	0.000	0.000	0.000	–
random	–	1 084	118	0.000	0.000	0.000	–

Table 6: 4×4 board after 300 000 training games (last row: one million games, two other seeds): mean over 2 seeds of the evaluation on 1 000 games each (95% half-width of the score about 2 000 for the best agents, of the probabilities at most 0.03). The greedy heuristic reaches 512 in 12% of the games and 1024 in 0.1%; the corner rule 512 in 5%.

Results. Table 6 and Figure 9 show that the semi-gradient Sarsa agent learns to play the real game from scratch, with no knowledge of the spawn distribution and no exploration other than the random tiles. The best configuration (6-tuples, $\alpha = 0.1$) reaches a mean score of 66 000, creates the 2048 tile in 90% of the games, 4096 in 67% and 8192 in 15%, in games of 3 000 moves on average; the hand-made heuristics never pass 1024 and score twenty times less. Three observations:

- **Features.** The 4-tuple network learns fast (the share of games reaching 2048 passes 50% within 10 000 games) but saturates around 45 000 points and 80% of 2048: rows, columns and

squares cannot represent the long-range structure (a monotone "snake" of large tiles) that the end-game needs. The 6-tuples start slowly, because their 33 million weights must each be visited, overtake the 4-tuples after 30 000 games, and are still improving at 300 000 (Figure 9b): a longer run of this configuration is reported below. This is the same approximation-error effect measured exactly on the small boards, with the roles reversed: here the larger table is needed and affordable.

- **Step size.** The learning rate trades speed against the final level: $\alpha = 0.5$ learns fastest in the first thousand games and ends worst (the 4-tuple agent with $\alpha = 0.5$ stalls at 19 000 points, with the TD errors of the long late games undoing what the early games learned); $\alpha = 0.05$–0.1 ends best. The effective per-weight step of [12], 0.0025 with 17 active features, corresponds to $\alpha \approx 0.04$ in our normalisation, consistent with this.
- **Comparison with the literature.** [12] report 97% of games reaching 2048 after 500 000 games with the 17 4-tuples and no weight sharing, and the stronger networks of [14, 5] reach 32768 regularly; our 300 000-game runs with the same update are in the expected range, and the gap to the state of the art is a matter of tuple sets, training time and step-size schedules, not of method.

Longer training. The best configuration (6-tuples, $\alpha = 0.1$) was trained for one million games with two further seeds (last row of Table 6; the curve of Figure 9b extends to 10^6). The agent keeps improving all the way: the mean score rises from 66 000 at 300 000 games to 92 000, the 2048 tile is reached in 93% of the games, 4096 in 81%, 8192 in 42% and 16384 in two games out of a thousand, in games of 4 000 moves; the two seeds agree within 3 000 points. Between 700 000 and one million games the share of games reaching 4096 still gains four points, so at this scale the 6-tuple network is limited by training time, not by its capacity, and the 97% of [12] at 500 000 games (a smaller per-weight step and no weight sharing, hence 17 independent tables) is within reach of the same code. The 33 million weights of the network are the price: each is updated only when its pattern appears, and the end-game patterns with large tiles appear only in long games, late in the training.

6.5 Final test: deep Q-learning

As a final test, outside the methods of the course, the RL part ends with a deep Q-network (DQN) [8] built as in the PyTorch tutorial [9]. DQN is Q-learning (§6.5) with a neural network approximating the action values (§9.7), stabilised by two devices that the course does not cover: an *experience replay memory*, from which every update draws a random minibatch of past transitions, and a *target network*, a slowly updated copy of the network that computes the targets $r + \gamma \max_{u'} Q'(x', u')$. Network and hyper-parameters are those of the tutorial: a perceptron with two hidden layers of 128 ReLU units and one output per move; a memory of the last 10 000 transitions and minibatches of 128; the Huber loss; AdamW with learning rate $3 \cdot 10^{-4}$ (amsgrad) and gradients clipped at 100; a soft update of the target network after every move, $\theta' \leftarrow 0.005\,\theta + 0.995\,\theta'$; ϵ-greedy exploration with $\epsilon = 0.01 + 0.89\,e^{-k/2500}$ after k moves; $\gamma = 0.99$; one gradient step per move. Only three things are adapted to 2048: the input is the one-hot encoding of the exponent of every cell (16 values per cell); the illegal moves are masked, both when acting and in the max of the target; the reward is the score of the move divided by the target tile (64 or 2048), so that the returns are of order one, which leaves the greedy policy unchanged. Unlike the agents of Sections 6.3 and 6.4, the network does not use the slide rule: like the plain tabular Q-learning, it learns $q(x, u)$ for the four moves from the board alone, and it must also discover which patterns of the board matter. It is trained with the score reward on the 2×3 board with target 64, where its greedy policy is evaluated exactly, and on the original 4×4 game, evaluated by simulation, with three seeds each.

	Games	Moves	Time	Greedy policy
DQN, 2×3, target 64	20 000	0.84 M	21 min	exact $P(\text{win}) = 0.812$ (0.807–0.818), score 408, optimal decisions 66%
Q-learning, tabular	20 000 / 200 000			$P(\text{win}) = 0.352$ / 0.757
Sarsa on afterstates, tabular	20 000 / 200 000			$P(\text{win}) = 0.648$ / 0.796
optimal policy (DP)				$P(\text{win}) = 0.939$, score 480
DQN, 4×4	10 000	1.50 M	48 min	score 2 314 (2 221–2 461); 256 in 57%, 512 in 4%, never 1024
Sarsa, n-tuple features	10 000			score 16 252 (6-tuples) / 30 972 (4-tuples); 2048 in 15% / 66%
corner / greedy rule				score 2 564 / 3 027; 512 in 5% / 12%

Table 7: Final test: DQN against the methods of the course, score reward (mean over 3 seeds; time per seed on one CPU core). 2×3 board: exact evaluation of the greedy policy; the tabular methods with $\epsilon = 0.1$, $\alpha = 0.1$ (Table 4). 4×4 board: 1 000 games with a fixed seed for the DQN and the n-tuple agents ($\alpha = 0.1$), 10 000 games for the heuristics.

Results. Table 7 and Figures 10–11 summarise the runs. On the 2×3 board the DQN is the fastest learner of the project: its greedy policy wins 0.69 of the games after 2 000 games and 0.812 (0.807–0.818 over the seeds) after 20 000, with an exact expected score of 408 out of 480, while after the same 20 000 games the tabular Q-learning with the score reward is at 0.35 and the Sarsa on afterstates at 0.65. It ends above all the tabular methods with the score reward after 200 000 games (at most 0.796) and below the best runs with the win reward and optimism (0.875). Two things make it fast: the network generalises across boards, so a game also improves the values of boards it has not visited, and the replay memory reuses every move in about 128 updates (one minibatch of 128 per move, from a memory of 10 000 moves), far more computation per game than a tabular update. It does not reach the optimum either: its curve flattens at 0.78–0.81 after 10 000 games, with 66% of optimal decisions, because a network of this size cannot represent V^* exactly and because it maximises the discounted score ($\gamma = 0.99$), not the win.

On the 4×4 board the picture is reversed. After 10 000 games (1.5 million moves, 48 minutes per seed) the DQN scores 2 314 on average, reaches 256 in 57% of the games and 512 in 4%, and never 1024: it has left random play (1 084) behind but not yet the corner rule (2 564) and the greedy rule (3 027), although its learning curve is still rising (Figure 11). After the same 10 000 games, semi-gradient Sarsa with n-tuple features scores 16 000 (6-tuples) to 31 000 (4-tuples) and reaches 2048 in 15–66% of the games, with a training time of minutes. The difference is in what each agent is given. The n-tuple agent receives the slide rule, through $\hat{q}(x, u) = r(x, u) + \hat{V}(M(x, u))$, and features that already describe the board in the terms that matter (the patterns of rows, columns and blocks of cells). The DQN must learn from 16 one-hot cells both the effect of every move and the useful features of the board, with a memory holding only the last 10 000 moves (about 50 games) and a discount that looks about 100 moves ahead in games of 200 moves and more. A check with the rewards divided by 64 or 256 instead of 2 048 (one seed, 2 000 games, `exp_dqn.py` with `scale=64` or `scale=256`) learns a little faster at the start (1 407 and 1 452 points against about 1 100) without changing the picture. The final test confirms on the real game what the small boards suggest: a generic deep network learns 2048, and on a small board it learns faster than a table, but on the large board the structure built into the course method, the afterstate parametrisation and the n-tuple features, is worth more than the extra capacity of the network.

7 The game itself: what is known

The questions a player asks about 2048 are the questions of this section: can the game always be won, is there an optimal strategy, and what are the good heuristics. The exact results of

Section 5.4 answer them completely for the small boards; for the original game the answers come from the literature.

Is 2048 always solvable? Not on the small boards: on the 2×3 board with target 64 even the optimal player loses 6% of the games, and as soon as $p_4 > 0$ there are unlucky spawn sequences that no policy survives (Figure 3b). On the 4×4 board the question is open. Slizkov [10] gives the only rigorous results: a strategy that reaches the 256 tile with probability 1, and a strategy that creates the 2048 tile with probability at least 0.99969. Whether some strategy wins with probability 1 is not known. If the new tiles were placed by an adversary instead of at random, the game would be lost well before 2048: this is easy to believe and to observe with "evil" variants of the game, but we found no proof of it. The largest tile that can ever appear on a 4×4 board is $131072 = 2^{17}$ (sixteen cells holding $2^{17}, 2^{16}, \ldots, 2^2$ plus a lucky final 4), and the best programs do reach $2^{15} = 32768$ regularly.

Is the optimal strategy known? No, and there are reasons to believe it is out of reach. The state space of the 4×4 game is bounded by $11^{16} \approx 4.6 \cdot 10^{16}$ states before the target (Table 2), which is beyond exact dynamic programming by many orders of magnitude; the exact solutions of this project stop at 3×3 boards with 5 million states. Computational complexity results show that the difficulty is intrinsic to the rules and not to the board size: Langerman and Uno [6] prove that deciding whether a given position on an $m \times n$ board can be played to reach a given tile value is NP-hard for 2048, Threes! and most of their variants; Abdelkader, Acharya and Dasler [1] prove NP-hardness even when no new tiles appear; Mehta [7] shows that reaching a target configuration is PSPACE-complete for a variant of the game. These are statements about generalised boards and worst-case instances, not about the probability of winning the 4×4 game, but they rule out a simple closed-form optimal strategy.

Which heuristics are good? Human players use the corner rule (keep the largest tile in a corner and the big tiles along one edge, ordered as a "snake"), keep the board monotone and smooth, and keep as many empty cells as possible. Our exact results qualify this advice: on the small boards the corner rule is dramatically worse than optimal play (0.07% versus 93.9% on $2 \times 3/64$, 36% versus 99.999% on $2 \times 4/64$, 43% versus 100% on $3 \times 3/64$), because a rigid move order wastes the few cells available. On the 4×4 board, programs built on exactly these heuristics do well when they are combined with search: the expectimax player of [13], with a hand-tuned evaluation (monotonicity, smoothness, empty cells, merges) and a depth of 4–8 plies, reaches 2048 in essentially every game and 16384 in most. The strongest players learn the evaluation instead of hand-tuning it: temporal-difference learning of the values of the afterstates with n-tuple features [12], the update of Section 6.4, wins 97% of the games after half a million training games; multistage learning and larger tuples [14] reach the 32768 tile in 31.75% of the games; Jaśkowski [5] adds temporal coherence learning, weight promotion and redundant encoding, and [4] optimistic initialisation, to reach 32768 in about 72% of the games with average scores above 600 000. All of these are, at the core, the semi-gradient update of this project: the state of the art of 2048 is a course method applied with care and a lot of computation.

8 Discussion and conclusions

What dynamic programming taught us about the game. The tile sum is a clock, so 2048 on any board is a total-cost problem with termination states that every policy reaches in a bounded number of moves: value iteration is exact after finitely many iterations (46 on the 2×3 board), and ordering the updates by tile sum turns it into a single backward pass over the reachable states. On the boards where this pass is affordable (up to 3×3, five million states) the exact values overturn the folk wisdom of the game: the corner rule, the greedy rule and random

play win a fraction of a percent of the games on $2 \times 3/64$ and $2 \times 4/128$ where the optimal player wins 94% and 99.7%, and the optimal policy agrees with the corner rule in only half of its decisions. Table 3 shows why: on a cramped board the value of a position is governed by whether a merge survives the next spawn, which a fixed move order ignores. Luck bounds every player: as soon as $p_4 > 0$ the optimal win probability is below one (Figure 3b), and the sensitivity to p_4 is large. Finally, the exact state counts (Table 2) make the curse of dimensionality concrete: 4×4 is four orders of magnitude beyond the last solvable board, and the complexity results of Section 7 say that no shortcut is to be expected.

Dynamic programming as a measuring instrument for reinforcement learning. The second use of the exact solution was to evaluate every learned policy exactly instead of by simulation, and every learned value function against V^* under the optimal occupancy. The learning curves of Sections 6.2–6.4 therefore have no Monte Carlo noise: differences of a few hundredths between methods are real, and the only randomness is the learning itself. This made four things visible that simulation would have blurred: on this game Monte Carlo prediction beats TD(0) and intermediate n beat both, the reverse of the textbook random walk at the start; the plateau of the n-tuple features (0.09 of win probability on 2×3 with rows, columns and squares, independent of the training length); the fact that symmetric weight sharing improves the values four-fold without improving the policy; and the gap between a learned Q and the greedy policy it induces (0.6 agreement with π^* and 0.8 of the optimal win probability at the same time).

What worked in reinforcement learning, and why. With a dense reward (the score) every tabular method of the course learns a reasonable policy on 2×3 within 200 000 episodes; with the sparse win reward and zero initial values the greedy learner locks on the first rewarded moves, and optimistic initialisation, not ϵ-greedy play, is what repairs it, because it explores every move once and then stops. Q-learning was the best method on plain tables (0.88 with optimism) and Sarsa on afterstates the best method overall (0.85 with decaying schedules, 0.92 with optimistic greedy play), for a structural reason: writing $q(x, u) = r(x, u) + V(M(x, u))$ is exact, one value per afterstate replaces four action values, the agent needs the slide rule but not the spawn distribution, and every update improves all the moves that lead to the same afterstate. Larger constant step sizes won within a fixed budget and decaying schedules only helped the methods that propagate credit further per update, the same trade-off that made intermediate n best in prediction. On the 4×4 board episodic semi-gradient Sarsa [11, §10.1] with linear n-tuple features reached 2048 in 90% of the games after 300 000 games and in 93% after a million, with a mean score of 92 000, thirty times the best heuristic, and no exploration other than the random tiles; the published players that reach 32768 use the same update with larger tuples, more games and better step-size control, so what separates this project from the state of the art is engineering, not method.

The final test. The deep Q-network of the PyTorch tutorial, outside the course, is the best learner on the 2×3 board for a given number of games (0.81 after 20 000 games, against 0.65 for the best table at the same point), because the network generalises and the replay memory reuses every move many times. On the 4×4 game, after 10 000 games, it is still below the rules of thumb (2 300 against 2 600–3 000 points), where the semi-gradient Sarsa with n-tuple features is already at 16 000–31 000. On this game, knowing the slide rule and choosing good features count for more than a deep network trained from raw cells.

What remains open. Whether the original game can be won with probability one is unknown (the best lower bound is 0.99969), the optimal policy is out of reach of exact methods and the problem is NP-hard on general boards, so the strongest strategies are learned approximations, validated by play rather than by proof. Within this project and the methods of the course, the natural next steps are semi-gradient n-step Sarsa [11, §10.2] with the same features, which the

prediction results suggest would learn faster than the one-step update, longer training of the 6-tuple features, and a direct measure, on the small boards, of how far the plateau of the features is from the best achievable with them (the projection of V^* on the feature space), which the exact solution makes possible.

A Reproducibility

The code is at `https://github.com/EdoardoCaproni/2048` (Python 3.10 or later, numpy, matplotlib, numba; PyTorch for the final test). The modules are `game2048.py` (rules of the game and the Gymnasium-style environment; the reference implementation), `dp2048.py` (exact DP: reachable states by stage, backward recursion, value iteration, policy evaluation, occupancy, Monte Carlo simulation), `fast2048.py` (the same rules on a 64-bit bitboard compiled with numba, about 50 times faster, checked move by move against the reference implementation by `test_2048.py`), `predict2048.py` (TD(0), n-step TD and Monte Carlo prediction), `rl2048.py` (tabular control agents and the exact evaluator), `ntuple2048.py` (semi-gradient Sarsa with n-tuple features) and `dqn2048.py` (the DQN of the final test). Every number and figure of this report is produced by a script in `experiments/` and written to `results/`:

Script	Produces
`dp_figures.py`	Section 5.4: Tables 2, 3, Figures 2a–3b
`dp_value_iteration.py`	Section 5.1: Table 1, Figure 1 (2 minutes)
`exp_prediction.py`	Section 6.2: Figure 4 (50 runs, under a minute)
`exp_tabular_2x3.py`, `plot_tabular.py`	Section 6.3: Table 4, Figure 5 (90 runs, 45 min on 26 cores)
`exp_tabular_grid.py`, `plot_schedule.py`	Figures 6, 7 (130 runs)
`exp_ntuple_small.py`, `plot_ntuple_small.py`	Section 6.4, small boards (41 runs, then the exact evaluations)
`exp_4x4.py`, `eval_4x4.py`, `plot_4x4.py`	Table 6, Figure 9 (14 runs of one hour)
`exp_dqn.py`, `plot_dqn.py`	Section 6.5: Table 7, Figures 10, 11 (6 runs in parallel, 50 min)
`export_viewer.py`	data of the policy viewer (best policy of every method, retrained)
`bench_engine.py`	throughput of the two engines

The notebook `2048_DP.ipynb` walks through Part 1 interactively, and `viz2048.py` animates any of the policies playing. The page `viewer/index.html`, which opens in any browser, shows the best policy of every method of this report playing: on the 2×3 board each policy is a table over the decision states, retrained with the configuration and the seed of its best run (the training is deterministic, so its exact win probability is the one reported), and the page shows at every move the optimal win probability V^* and the exact value Q^* of every move; on the 4×4 board it runs the n-tuple agent (4-tuples) and the DQN.

References

[1] Ahmed Abdelkader, Aditya Acharya, and Philip Dasler. 2048 without new tiles is still hard. In *8th International Conference on Fun with Algorithms (FUN 2016)*, volume 49 of *LIPIcs*, pages 1:1–1:14, 2016. Preprint: On the complexity of slide-and-merge games, arXiv:1501.03837.

[2] Dimitri P. Bertsekas. *Reinforcement Learning and Optimal Control*. Athena Scientific, 2019.

[3] Gabriele Cirulli. 2048. `https://github.com/gabrielecirulli/2048`, 2014.

[4] Hung Guei. *On Reinforcement Learning for the Game of 2048*. PhD thesis, National Yang Ming Chiao Tung University, 2022. arXiv:2212.11087.

[5] Wojciech Jaśkowski. Mastering 2048 with delayed temporal coherence learning, multistage weight promotion, redundant encoding, and carousel shaping. *IEEE Transactions on Computational Intelligence and AI in Games*, 10(1):3–14, 2018. arXiv:1604.05085.

[6] Stefan Langerman and Yushi Uno. Threes!, Fives, 1024!, and 2048 are hard. In *8th International Conference on Fun with Algorithms (FUN 2016)*, volume 49 of *LIPIcs*, pages 22:1–22:14, 2016. Journal version: Theoretical Computer Science 748, 17–27, 2018.

[7] Rahul Mehta. 2048 is (PSPACE) hard, but sometimes easy, 2014. arXiv:1408.6315.

[8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[9] Adam Paszke and Mark Towers. Reinforcement learning (DQN) tutorial. PyTorch Tutorials, `https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html`. Accessed November 2026.

[10] Alexey Slizkov. Computational bounds for the 2048 game, 2023. arXiv:2303.07266.

[11] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[12] Marcin Szubert and Wojciech Jaśkowski. Temporal difference learning of N-tuple networks for the game 2048. In *IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8, 2014.

[13] Robert Xiao. 2048 AI: expectimax optimisation with a bitboard engine. `https://github.com/nneonneo/2048-ai`, 2014.

[14] Kun-Hao Yeh, I-Chen Wu, Chu-Hsuan Hsueh, Chia-Chuan Chang, Chao-Chin Liang, and Han Chiang. Multistage temporal difference learning for 2048-like games. *IEEE Transactions on Computational Intelligence and AI in Games*, 9(4):369–380, 2017.

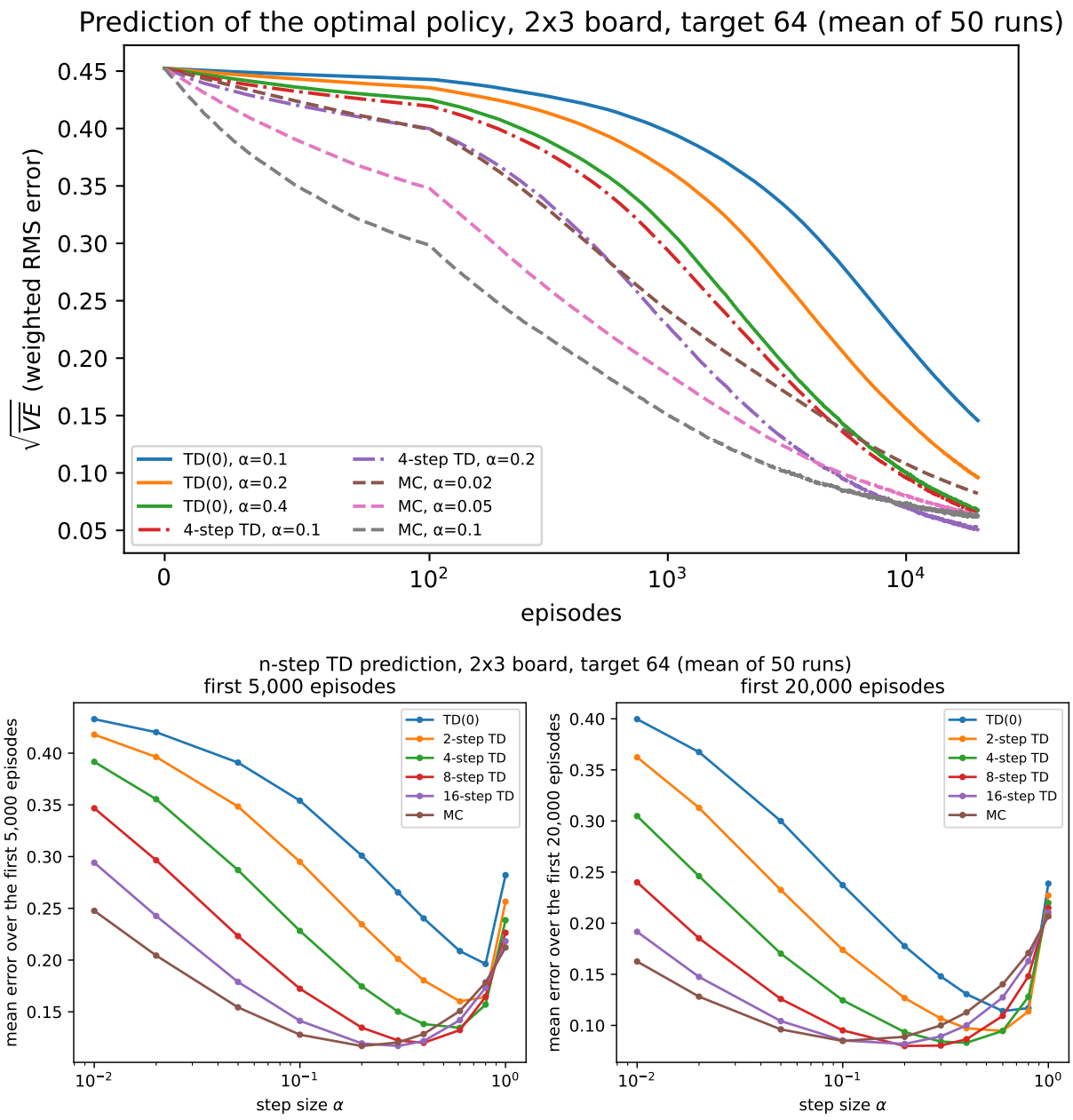
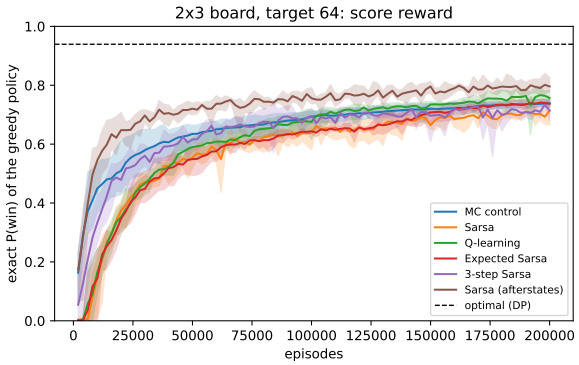
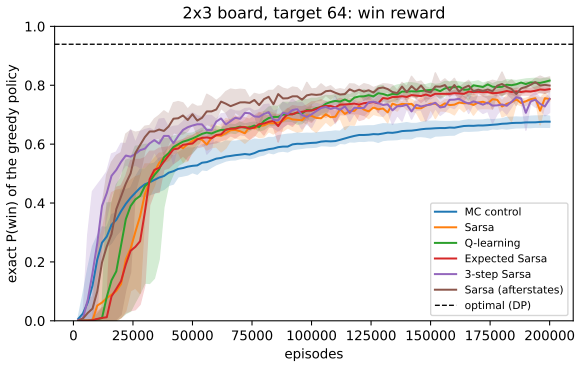


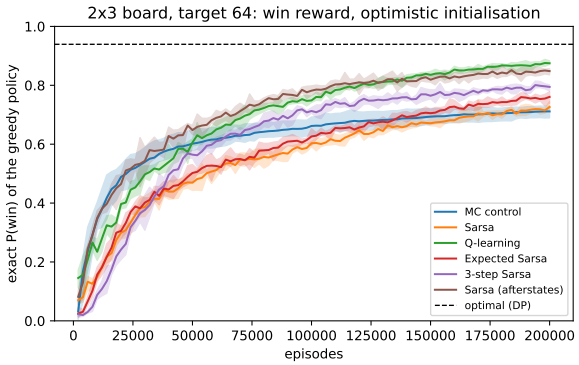
Figure 4: Prediction of the win probability of the optimal policy, 2×3 board, target 64, mean of 50 runs. Top: error against the number of games (logarithmic scale beyond 100). Bottom: error averaged over the first 5 000 and 20 000 games as a function of the step size, one line per n (MC: Monte Carlo); compare [11, Figure 7.2].



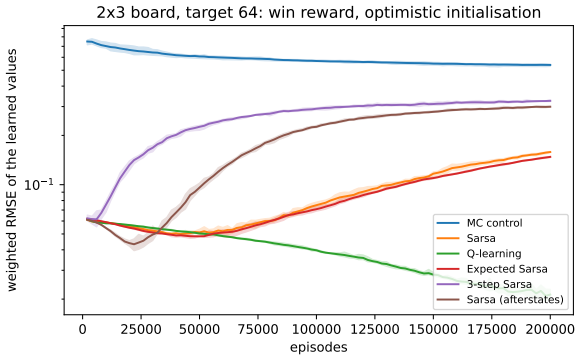
(a) Score reward.



(b) Win reward, zero initial values.



(c) Win reward, optimistic initial values.



(d) Value error, win reward, optimistic initial values.

Figure 5: Tabular methods on the 2×3 board, target 64: exact win probability of the greedy policy (mean over 5 seeds, band = min–max) and weighted RMSE of the learned values.

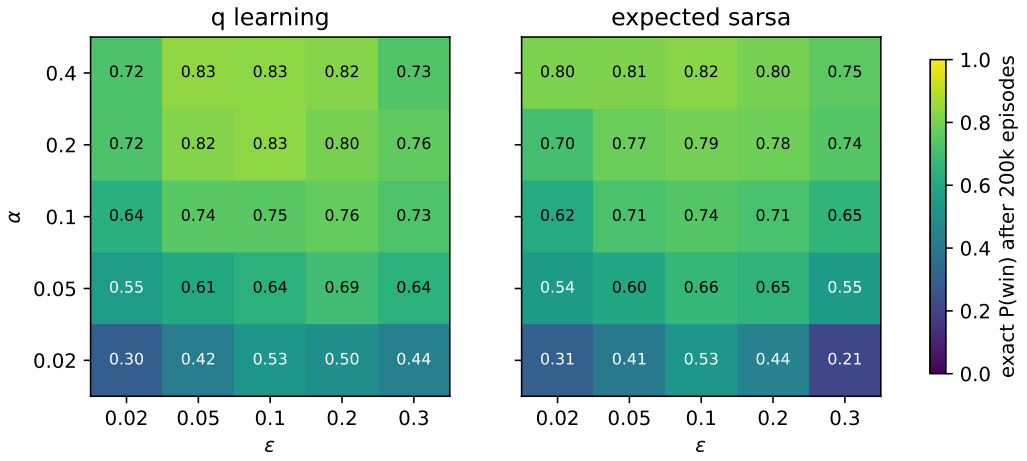


Figure 6: Exact win probability of the greedy policy after 200 000 episodes with the score reward, as a function of the constant step size α and exploration rate ϵ (mean over 2 seeds).

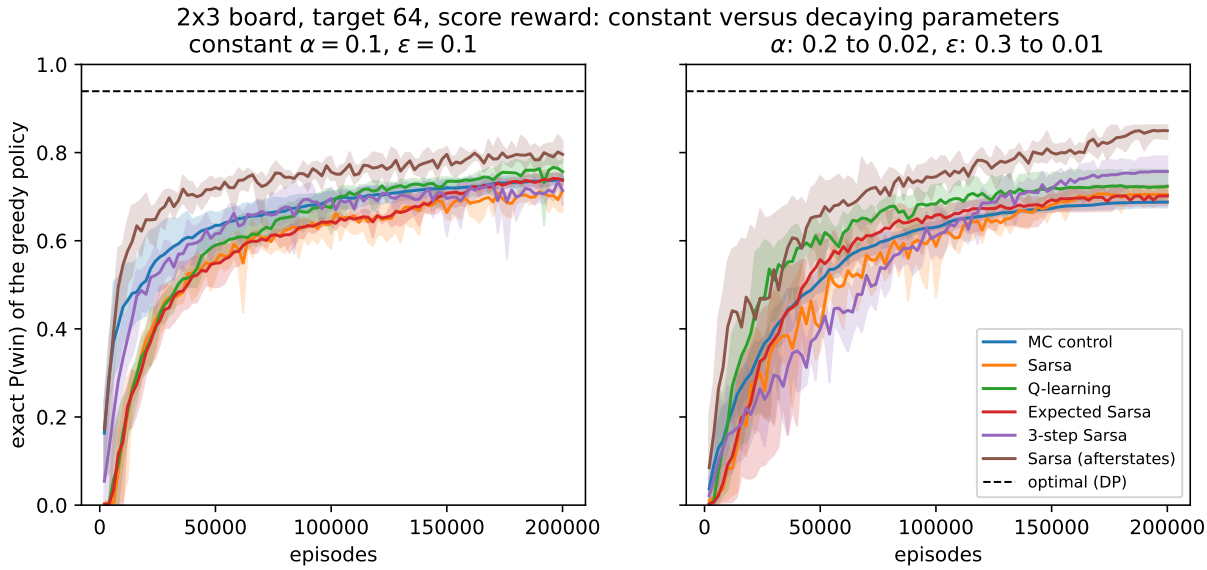


Figure 7: Constant versus decaying α and ϵ, score reward, exact win probability of the greedy policy (mean over 5 seeds, band = min–max).

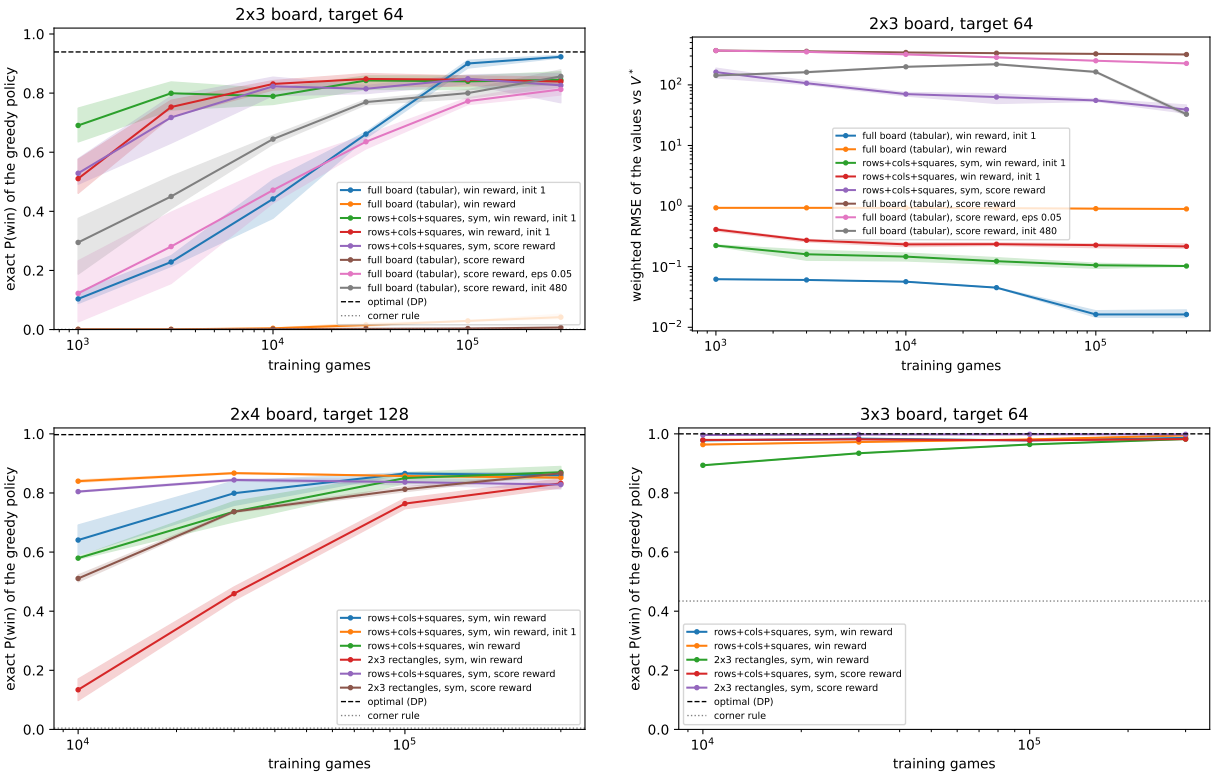
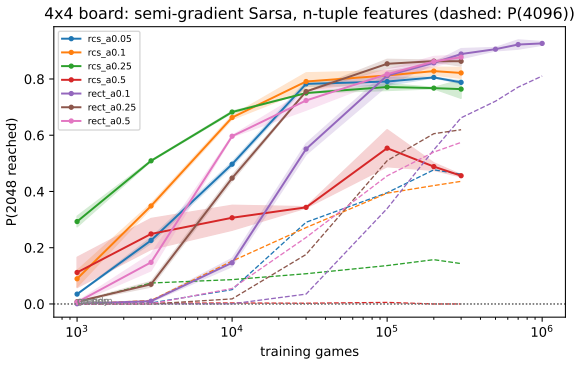
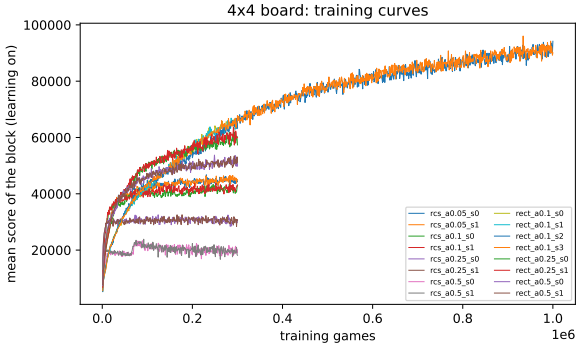


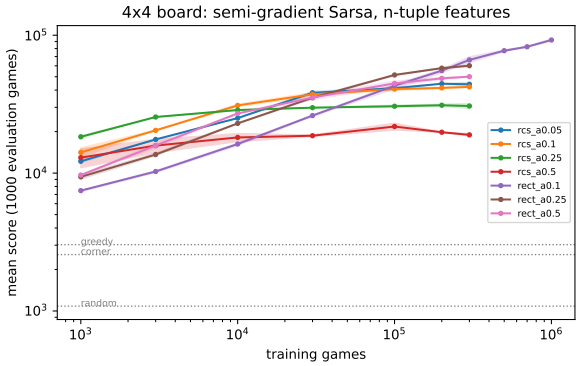
Figure 8: n-tuple networks on the small boards (mean over seeds, band = min–max). Top: 2×3 board, target 64, exact win probability of the greedy policy and weighted RMSE of the learned values against V^* (for the score reward the values are scores, in the range 0–480). Bottom: 2×4 board, target 128, and 3×3 board, target 64. Dashed: optimal; dotted: corner rule.



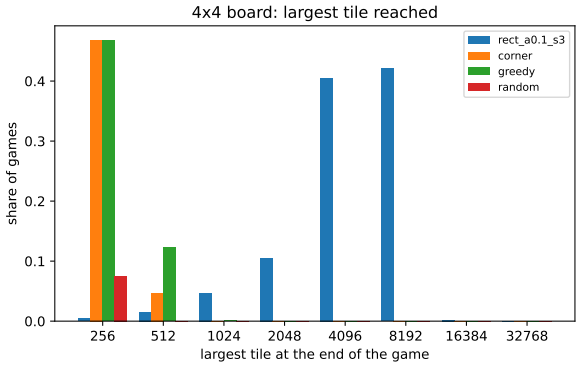
(a) Share of evaluation games reaching 2048 (solid) and 4096 (dashed).



(b) Mean score of each block of 1 000 training games.



(c) Mean evaluation score (log scales).



(d) Largest tile at the end of the game, best agent and heuristics.

Figure 9: Semi-gradient Sarsa agents with n-tuple features on the 4×4 board. rcs = rows, columns and squares (4-tuples), rect = 2×3 rectangles (6-tuples); bands = min–max over 2 seeds.

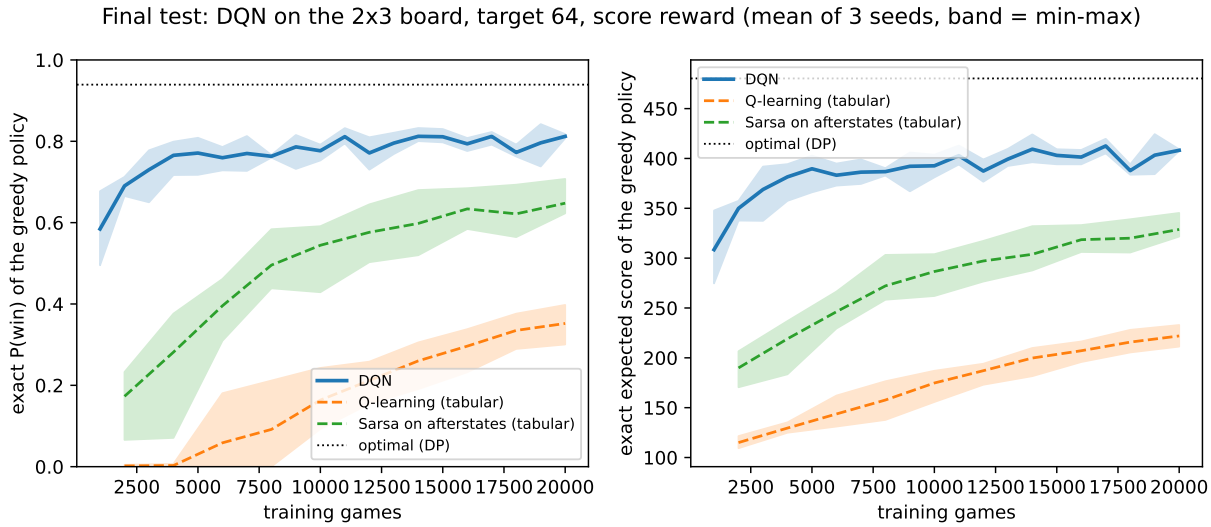


Figure 10: Final test on the 2×3 board, target 64, score reward: exact win probability and expected score of the greedy policy of the DQN against the tabular Q-learning and Sarsa on afterstates in their first 20 000 games (mean of 3 and 5 seeds, band = min–max).

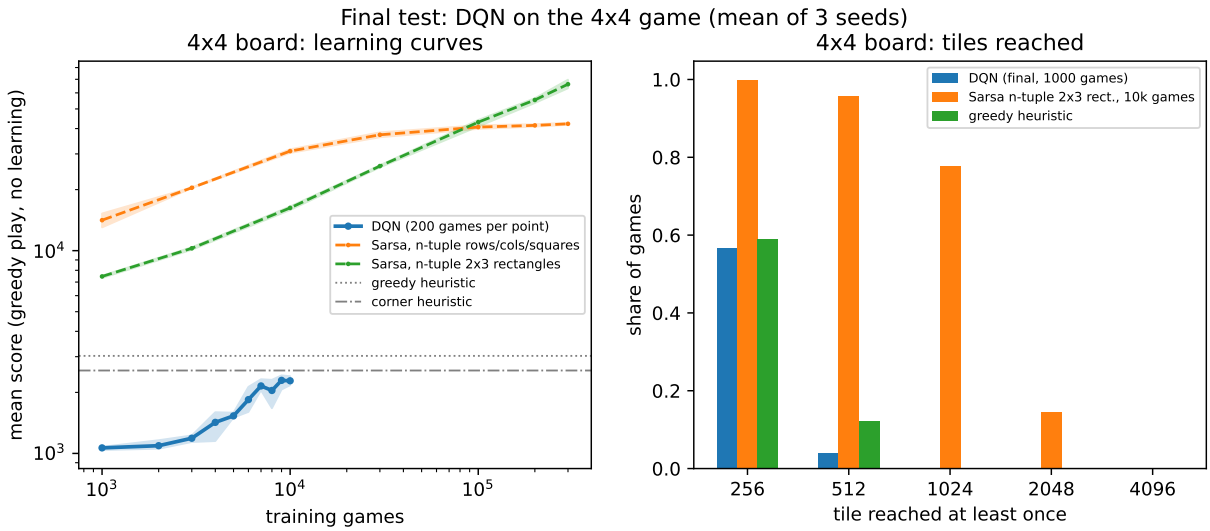


Figure 11: Final test on the 4×4 game. Left: mean score of the greedy policy against the training games, DQN (200 games per point) and semi-gradient Sarsa with n-tuple features (1 000 games per point), with the two rules of thumb. Right: share of the evaluation games reaching each tile.